

Expt. No. : 6

Date :

CAR CLASS AND AGE CALCULATION

AIM

To write a Python program that creates a class with attributes and methods to read input from the user and display the information.

SOFTWARE REQUIRED

Python 3.14

THEORY

CLASSES IN PYTHON:

In Python, a class is a user-defined data type that contains both the data itself and the methods that may be used to manipulate it. classes serve as a template to create objects. They provide the characteristics and operations that the objects will employ.

For example, a class is a prototype of a building. A building contains all the details about the floor, rooms, doors, windows, etc. we can make as many buildings as we want, based on these details. Hence, the building can be seen as a class, and we can create as many objects of this class.

CREATING CLASSES IN PYTHON

In Python, a class can be created by using the keyword class, followed by the class name. The syntax to create a class is given below.

Syntax

```
class ClassName:
    #statement_suite
```

OBJECTS IN PYTHON:

An object is a particular instance of a class with unique characteristics and functions. After a class has been established, you may make objects based on it. By using the class constructor, you may create an object of a class in Python. The object's attributes are initialised in the constructor, which is a special procedure with the name `__init__`.

Syntax:

```
# Declare an object of a class
object_name = Class_Name(arguments)
```

The self-parameter

The self-parameter refers to the current instance of the class and accesses the class variables. We can use anything instead of self, but it must be the first parameter of any function which belongs to the class.

`__init__` method

In order to make an instance of a class in Python, a specific function called `__init__` is called. Although it is used to set the object's attributes, it is often referred to as a constructor.

The self-argument is the only one required by the `__init__` method. This argument refers to the newly generated instance of the class. To initialise the values of each attribute associated with the objects, you can declare extra arguments in the `__init__` method.

PROBLEM

1. Build a Python program to create a class called `MobilePhone` with attributes `brand`, `model`, and `battery_percentage`. Implement a method to display the battery status.
2. A power generation station records the operating details of generators. Build a Python program to create a class called `Generator` with attributes `generator_name`, `rated_power`, and `operating_hours`. Implement a method to calculate the total electrical energy generated during operation.
3. For this challenge, you are given two complex numbers, and you have to print the result of their addition, subtraction, multiplication, division and modulus operations. The real and imaginary precision part should be correct up to two decimal places.

ALGORITHM

STEP 1: Start the program.

STEP 2: Read the car company, model, manufacturing from the user using the input,

function and assign it to variable 'company', 'model', 'year'.

STEP 3: Create an instance of the car class with the input 'company', 'model', 'year'.

STEP 4: Call the calculate_age() method of the car instance to calculate the age of car

STEP 5: Print the age of car along with the company, model using the print function

STEP 6: Terminate the program.

PROGRAM CODING

```
ratings = []
```

```

n = int(input("Enter how many transformer ratings: "))
for i in range(n):
    rating = int(input("Enter transformer rating: "))
    ratings.append(rating)
capacity = int(input("Enter maximum capacity: "))
result = []
for i in ratings:
    if i > capacity:
        result.append(i)
print("Transformers greater than capacity:", result)

```

OUTCOMES OF PROGRAM CODING

```

IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
==== RESTART: C:\Users\Mugil\Desktop\Python\Experiment\Experiment6.py =====
Enter Mobile Brand: Samsung
Enter Mobile Model: S24
Enter Battery Percentage: 85
Phone: Samsung S24
Battery Status: 85.0%
>>>

```

PROBLEM CODING

```

ratings = []

```

```

n = int(input("Enter how many transformer ratings: "))
for i in range(n):
    rating = int(input("Enter transformer rating: "))
    ratings.append(rating)
capacity = int(input("Enter maximum capacity: "))
result = []
for i in ratings:
    if i > capacity:
        result.append(i)
print("Transformers greater than capacity:", result)

```

OUTCOMES OF PROGRAM CODING

```

IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:\Users\Mugil\Desktop\Python\Problem\experiment6.py =====
Enter Generator Name: motor-1
Enter Rated Power (in kW): 50
Enter Operating Hours: 8
Total Energy Generated by motor-1: 400.0 kWh

```

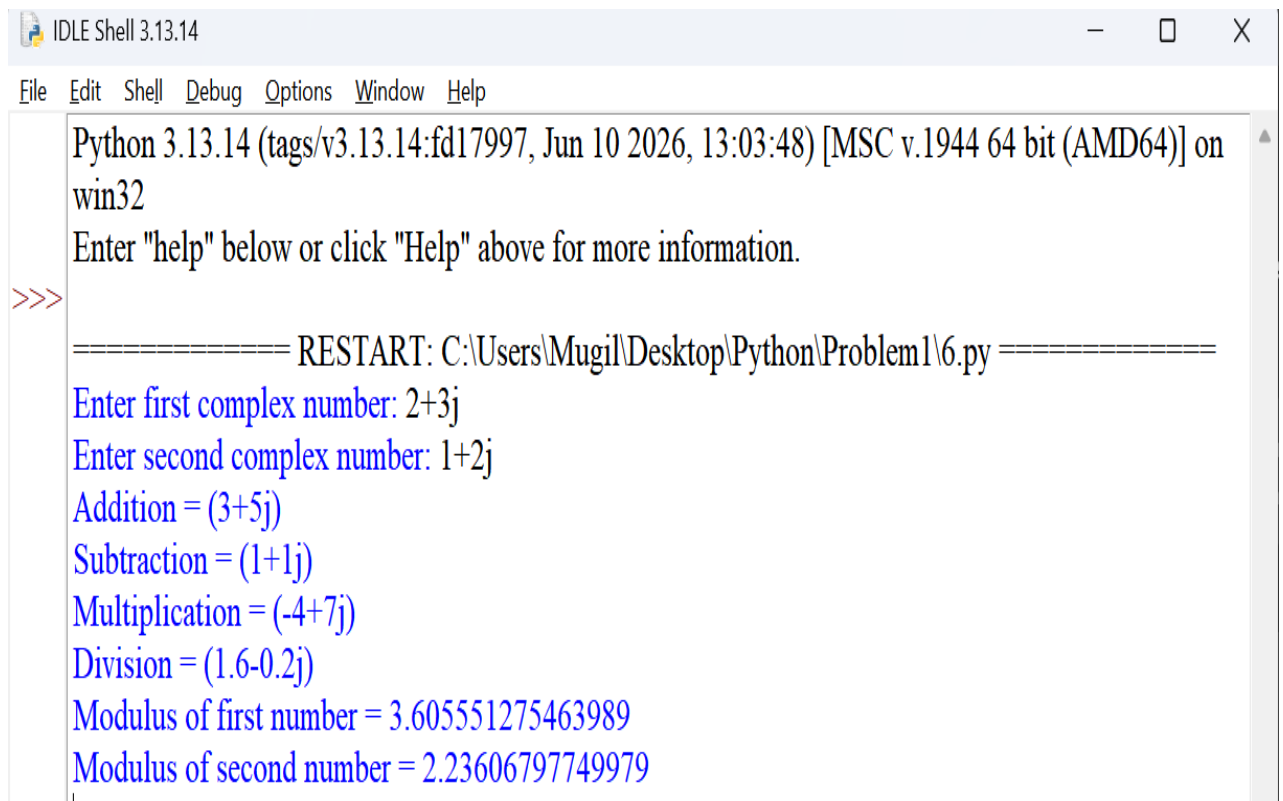
PROBLEM CODING

```

a = complex(input("Enter first complex number: "))
b = complex(input("Enter second complex number: "))
print("Addition =", a + b)
print("Subtraction =", a - b)
print("Multiplication =", a * b)
print("Division =", a / b)
print("Modulus of first number =", abs(a))
print("Modulus of second number =", abs(b))

```

OUTCOMES OF PROGRAM CODING



```

IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:\Users\Mugil\Desktop\Python\Problem1\6.py =====
Enter first complex number: 2+3j
Enter second complex number: 1+2j
Addition = (3+5j)
Subtraction = (1+1j)
Multiplication = (-4+7j)
Division = (1.6-0.2j)
Modulus of first number = 3.605551275463989
Modulus of second number = 2.23606797749979

```

VIVA QUESTION

1. What is a class in Python, and how do you define one?

A class in Python is a blueprint for creating objects (instances), defining their behavior (methods) and properties (attributes). You define a class using the class keyword followed by the class name.

2. What is the purpose of the `__init__` method in a class?

The `__init__` method is a special method used for initializing an object's attributes when it is created. It is called automatically when an object is instantiated.

3. How do you create an object of a class, and how do you access its attributes?

You create an object of a class by calling the class name as if it were a function. You can access the object's attributes using the dot `.` notation.

4. Explain the difference between instance attributes and class attributes.

- Instance attributes are unique to each object and defined inside the `__init__` method.
- Class attributes are shared across all instances of the class and defined directly within the class but outside any methods.

5. What is the self keyword, and why is it important in class methods?

The self-keyword refers to the instance of the class and is used to access instance attributes and methods from within class methods. It must be the first parameter of any instance method.

6. How do you implement inheritance in Python?

Inheritance is implemented by passing the parent class as a parameter to the child class. The child class inherits all attributes and methods of the parent class.

7. What is method overriding, and how is it implemented?

Method overriding occurs when a child class defines a method with the same name as a method in its parent class. This allows the child class to provide a different implementation for that method.

8. How do you use the `super()` function in a class?

The `super()` function is used to call a method from the parent class. It is commonly used to invoke the parent class's `__init__` method from within the child class.

9. What is the purpose of the `__str__` and `__repr__` methods in a class?

- a. `__str__`: Defines a human-readable string representation of an object, used when you print the object.
- b. `__repr__`: Defines an official string representation of an object, which is used for debugging and logging.

10. How do you define and use a class method and a static method?

- Class method: Defined using the `@classmethod` decorator and takes `cls` (the class itself) as its first parameter. It can modify class-level attributes.
- Static method: Defined using the `@staticmethod` decorator and does not take `self` or `cls` as a parameter. It behaves like a regular function inside a class and does not modify object or class state.

RESULT:

Thus, the Python program successfully created a class named Car with the attributes Company, Model, and Year. The program correctly calculated and

displayed the age of the car in years using a class method. The program was executed successfully, and the output was verified.

INFERENCE:

The experiment demonstrates that Python classes and objects can effectively model real-world entities by encapsulating data and behaviour. The successful implementation of the Car class and its method validate the principles of object-oriented programming, including class creation, object instantiation, attribute initialization, and method execution.

Assessment	Max. Marks	Marks Obtained
Conduction of Practical	15	
Report	5	
Viva	5	
Total Marks	25	
Signature of Lab In-Charge		